ATIVIDADE	PROJETO INTEGRADOR (PWBE)

CONTEXTO e DESCRIÇÃO

A escola **TecnoVille** está desenvolvendo um projeto de transformação urbana baseado no conceito de **Smart City**. A ideia é implementar sensores em pontos estratégicos da cidade para coletar dados em tempo real sobre:

- **Temperatura**
- **Umidade**
- **Luminosidade**
- **Contador**

Esses sensores serão instalados em locais como praças, corredores, pátios etc.

A gestão escolar contratou sua equipe técnica para desenvolver **o back end** do **sistema de monitoramento** usando **Django e Django Rest Framework**. Os dados dos sensores serão enviados para a API e posteriormente visualizados por um painel web.

Desafio proposto

Você deve **modelar e desenvolver o back end completo da aplicação**, com base nos seguintes requisitos:

Requisitos Funcionais

- O sistema deve permitir **registrar sensores**, com campos como:
  - ID do sensor
  - Sensor (temperatura, umidade, luminosidade, contador)
  - Identificação (mac-address)
  - Localização (latitude e longitude)
  - Status operacional (ativo/inativo)
- Deve ser possível **registrar medições** vindas dos sensores:
  - ID do sensor relacionado
  - Ambiente (relacionado com a tabela de ambientes)
  - Valor da medição
  - Data e hora da leitura (timestamp)
- Deve se registrar os **ambientes** da escola:
  - Local
  - Descrição
  - Responsável
- Deve se registrar os **locais** onde os sensores estarão localizados na escola:
  - Local
- Deve se registrar os **responsáveis** pelos ambientes da escola:
  - nome

- A API deve fornecer endpoints para:
  - Cadastrar, editar, listar e apagar sensores da tabela Sensores.
  - Cadastrar e listar medições por sensor
  - Listar as medições mais recentes (últimas 24h, por exemplo)

Requisitos do Projeto:

Back-End (Django Rest Framework):

Criação de uma API RESTful para gerenciar dados de sensores.

- A API deve ter endpoints para criar, ler, atualizar e deletar (CRUD) dados dos sensores e ambientes.

Os dados dos sensores devem incluir:

Temperatura (°C)

Luminosidade (lux)

Umidade (%)

Contador(num)

Os dados devem ser armazenados em um banco de dados MySQL.

Implementar autenticação utilizando JSON Web Tokens (JWT) para proteger os endpoints.

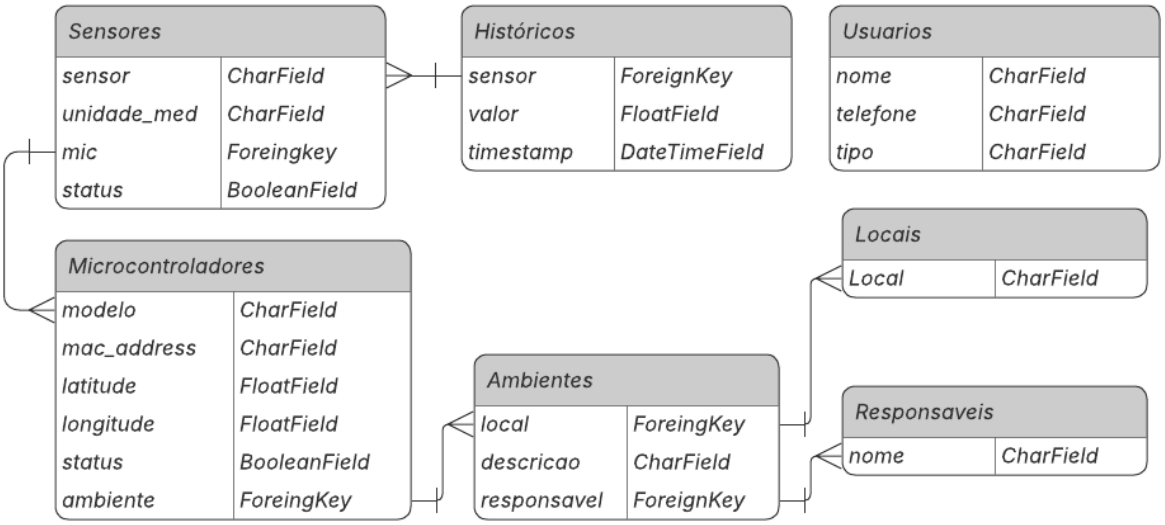

Figura 1 - Diagrama de Entidade e Relacionamento

- **Login**

  Criar um super usuário para o nosso api_smart.
  - **username = senai**
  - **password = 123**
- Tabela *Usuários*:
  - Na tabela **Usuários** o **tipo** deverá ter um CHOICES com: *Administrador* e *Usuário*, onde o primeiro poderá fazer o CRUD completo e o segundo somente visualizar os dados.
- Tabela *Sensores*:
  - Na tabela **Sensores**, para padronizar, no campo ***sensor***, recomenda-se criar um CHOICES com: *temperatura, umidade, luminosidade* e *contador* e no campo ***unidade_med***: ºC, %, lux, uni.

- **Relacionamento entre tabelas**
  - Os relacionamentos deverão ser aplicados nas tabelas conforme diagrama já mencionado acima.

- **Gerenciamento dos Sensores:**
  - Em todas as páginas os elementos deverão ser listados na forma de tabela.
- **Dados:**
  - Criar método para popular banco de dados de sensores, ambientes, locais e responsáveis que estão nas planilhas disponibilizadas.
- **Integração entre Front End e Back End:**
  - Consumir a API Django.
  - Inicie com uma página de **login**.
  - Ao logar direcione para a página **home** em que teremos os 4 sensores que devem ser botões ou links diretos para página específica de cada sensor.
  - Ao acessar a página do sensor específico deve-se listar os dados em forma de tabela.

Tarefas a Serem Realizadas:

- **Desenvolvimento do Back-End:**
  - **Histórias de Usuário:**
    - Como administrador, eu quero criar um endpoint para registrar dados de sensores, para armazenar os dados de temperatura, luminosidade, umidade e contador.
    - Criar um endpoint para visualizar os dados dos sensores, para poder monitorar as condições ambientais.
    - Implementar autenticação JWT, para garantir que apenas usuários autorizados acessem os dados.
  - **Tarefas:**
    - Configurar projeto Django e instalar o Django Rest Framework..
    - Criar modelos para dados de sensores.
    - Implementar serializers e views.
    - Configurar URLs e autenticação JWT.

- **Desenvolvimento do Front-End:**
  - **Histórias de Usuário:**
    - Como supervisor, eu quero visualizar os dados dos sensores em uma lista, para monitorar as condições em tempo real.
    - Realizar login na aplicação, para acessar os dados de forma segura.
  - **Tarefas:**
    - Configurar projeto em React, HTML, Angular ou outro.
    - Criar tela de login.
    - Conectar a aplicação à API utilizando axiod, fetch, httpClient ou outro.
    - Implementar autenticação JWT no front-end.

- **Testes e Simulação:**
  - Implementar scripts para gerar dados simulados de sensores.
  - Testar a API com ferramentas como Postman ou **Insomnia**.
  - Garantir que a aplicação exiba corretamente os dados simulados.

- **Funcionalidades Extras (Opcionais)**
  As funcionalidades abaixo **não são obrigatórias para aprovação**, porém:

  ✓ podem contribuir nos **critérios desejáveis**
  ✓ servem como **diferencial técnico do projeto**
  ✓ demonstram maior domínio do conteúdo

- **Lista de funcionalidades:**

  ✓ Paginação de dados no front
  ✓ Estilização da aplicação
  ✓ Página de cadastro de usuário
  ✓ Página de cadastro de sensores
  ✓ Página de entrada manual de dados para sensores
  ✓ Página de gráficos com períodos pré-definidos
  ✓ Visualização de gráficos para análise das variações dos sensores ao longo do tempo
  ✓ Inclusão de ícones de edição e exclusão nas listagens
  ✓ Exportação de dados no formato XLSX
  ✓ Implementação de regra de negócio: não permitir medições quando o sensor estiver inativo

- **Observações:**

  ✓ Os dados para popular o banco serão fornecidos pelo professor.
  ✓ O trabalho é **individual**.
  ✓ Na data de entrega, o professor realizará a correção diretamente no repositório, considerando o estado atual do projeto no momento da avaliação (não serão aceitas alterações posteriores).
  ✓ O repositório deverá ser **privado**, contendo apenas o acesso ao professor pelo e-mail: **lindomarbatistao@gmail.com**.
  ✓ Dúvidas devem ser relacionadas **exclusivamente à interpretação do enunciado**, não sendo permitida orientação técnica detalhada durante a execução do projeto.
  ✓ Em caso de **plágio ou cópia parcial/total**, todos os trabalhos envolvidos receberão **nota zero**, até que a situação seja devidamente analisada e esclarecida.
  ✓ Data de entrega: **16/04/2026**, podendo ser prorrogada, no máximo, até **17/04/2026**.

Critérios de avaliação

CRITÉRIOS CRÍTICOS (50 pontos — mínimo para funcionamento)

Nº	Critério	Nota
1	Estrutura do projeto Django + DRF configurado corretamente	4
2	Modelos criados corretamente (Sensor, Ambiente, Histórico, etc.)	6
3	Relacionamentos entre tabelas funcionando	4
4	Migrações aplicadas e banco funcionando	2
5	Serializers básicos funcionando	4
6	CRUD básico funcionando (Sensores e Ambientes)	8
7	Endpoint de medições (criar e listar)	6
8	Autenticação JWT funcionando (/token/)	6
9	Proteção básica de endpoints (IsAuthenticated)	4
10	Testes básicos via Postman/Insomnia	2
11	Front: login funcional consumindo API	2
12	Front: armazenamento do token	1
13	Front: consumo de pelo menos 1 endpoint (listagem)	1
	Total	**50**

CRITÉRIOS DESEJÁVEIS (50 pontos — nível avançado)

Nº	Critério	Nota
14	CRUD completo (todos os modelos) bem estruturado	5
15	Filtros avançados (data, status, sensor, local)	5
16	Endpoint de medições recentes (lógica correta)	4
17	Validações de regra de negócio (ex: sensor inativo não recebe dados)	4
18	Tratamento de erros HTTP (400, 401, 404 padronizados)	4
19	Organização do código (views limpas, separação de lógica)	4
20	Importação de dados (CSV/Pandas ou script automatizado)	4
21	Distinção de níveis de usuários (Admin pode tudo e User só visualiza)	3
22	Configuração e funcionamento do banco em MySQL	4
23	Front: proteção de rotas (sem token não acessa)	2
24	Front: tratamento de erro 401 (redireciona login)	1
25	Front: navegação funcional (login → listagem → detalhe)	1
26	Front: consumo de múltiplos endpoints (sensores + medições)	1
27	Front: envio de dados (POST funcionando)	1
28	Front: logout funcional	1
29	Código limpo e padronizado	1
30	Versionamento organizado (GitHub correto)	1
31	Domínio do projeto (pergunta oral)	4
	Total	**50**